A Neural Network Approach to Predicting the Approval or Rejection of a Credit Card Application

Cutines, Adrian Carl B.

CMSC 191: Special Topics: Fundamentals of Neural Computing

1. Abstract

Credit card providers are required to make timely and consistent decisions over whether to accept an application. A risk ranking model can help to make a decision. We have implemented a feedforward network of logistic units trained using the back propagation algorithm with adaptive learning rate and momentum as outlined in the course, in QB64. This model attempts to predict the outcome of 1,548 credit card applications based on 17 attributes, provided by applicants. Since there were duplicate applicants and only 11.3% of applications were rejected (the positive class), we grouped duplicates together when splitting into training, validation and testing sets. All features were scaled between 0 and 1 and equal weighting was given to both classes during training. When splitting the data into 5 different training/validation/testing sets, a model with 4 hidden nodes trained for 300 epochs had a mean AUC of 0.602 (sd = 0.061) across testing sets. This value was greater than 0.5 (random chance) across all runs. However, accuracy, precision, recall and F1 scores were highly dependent on the decision threshold. Overall this shows that a network created using the algorithm from the course can learn a weak but real signal from the data provided. Grouping duplicates and addressing the class imbalance are important considerations when interpreting the scores

2. Introduction

When assessing whether to approve an application for a credit card, there are costs associated with approving high risk candidates. However, there are also costs associated with turning down creditworthy applicants. To make systematic decisions, lenders will use a credit scoring system. Often, credit scoring is considered to be a classification problem (Baesens et al., 2003). Using neural networks to perform credit scoring is attractive because nonlinear relationships can be captured by using hidden logistic units. This small tabular dataset is a good candidate for demonstrating the concepts taught in this course, such as logistic units, back propagation and adaptive learning rates, can be used to build a predictive model.

Credit scoring using classification models, including neural networks has been researched for many years. It has been shown that many different classification models perform well on several real-world credit datasets (Baesens et al., 2003). This suggests that credit data is often only slightly nonlinear. There are four practical considerations when dealing with credit data. Firstly, credit data is highly imbalanced. Imbalance requires special consideration during training, such as using cost-sensitive error weights (He & Garcia, 2009). Secondly, networks trained using back propagation require consideration of practical factors such as scaling the input variables and learning rate (LeCun et al., 1998). Thirdly, the evaluation measure should be chosen appropriately. The area under the receiver operating characteristic curve (AUC) provides a summary of how well the classifier ranks positive examples over negative examples. This is

robust to the class balance of the data (Fawcett, 2006). Finally, if there are duplicate entries for the same individual in the training and test data then there may be overly optimistic scores. In this case it is best to ensure that all the data for one individual remains within one split.

We are looking at this problem from a more specific angle than most of the literature. Rather than trying to find the most accurate model, we are looking at what can be achieved by building a network using the algorithm provided by this course. The network uses the back propagation algorithm (Rumelhart et al., 1986) and adaptive learning rate rule from the course program. We have kept the network architecture simple, and looked at a few different configurations including number of hidden nodes, length of training and seed. This is because there are very few positive examples and using a large network may cause the network to overfit.

In this paper, we preprocessed the data for the logistic unit network. We grouped together duplicate applicants, so that, as far as our grouping rule could detect, no applicant appeared in both the training and test data . We trained the back propagation algorithm from the course (compiled in QB64) using class weighted errors. We looked at using a larger network and a smaller network, early stopping and fixed length training. We then ran the selected configuration 5 times and assessed the performance of the network based mostly on the AUC score from the test data.

The network was able to learn a weak signal. We found that our selected configuration gave us an average AUC score of 0.602 from 5 different splits. All five runs were better than random. The AUC varied quite a lot between runs (0.528-0.660). The accuracy, precision and recall varied greatly depending on the decision threshold. As there were very few positive examples in each test set, there is a large amount of uncertainty surrounding the performance. Therefore we have reported the average score from the splits.

3. Objectives

General objective: To design and evaluate a feedforward neural network, built on the course's backpropagation algorithm, that predicts credit card application outcomes from applicant records.

Specific objectives:

1. To prepare the credit application data so that it suits a logistic-unit network and so that evaluation reflects performance on applicants the network has not seen.
2. To train and compare network configurations (number of hidden nodes and training strategy) using the course backpropagation program.
3. To measure the predictive performance and the stability of the selected configuration across repeated data splits, using a threshold-independent metric (AUC) and threshold-dependent metrics.

4. Methodology

4.1 Data and preprocessing (Objective 1)

The data consist of two files joined on the applicant identifier: an inputs file with 17 attributes and an output file with a binary label. There are 1,548 applications, of which 175 (11.3%) have label 1, meaning the application was rejected (label 0 means approved), as defined in the dataset description. We call the rejected applications the positive class . Table 1 summarizes the attributes and how each was encoded.

Table 1. Attributes and encoding

Group	Attributes	Encoding	Inputs
Binary	GENDER, Car_Owner, Propert_Owner	0/1	3
Pass-through	CHILDREN, Family_Members, Work_Phone, Phone, EMAIL_ID, Mobile_phone	used as given (Mobile_phone is constant)	6
Numeric	Annual_income, Birthday_count, Employed_days	median imputation, then min-max scaling	3
Categorical	Type_Income, EDUCATION, Marital_status, Housing_type	one-hot encoding	20
Dropped	Ind_ID, Type_Occupation	not used	0
Total			**32**

All choices are based on properties of the network. Since both the hidden and output layers use logistic units which take values between 0 and 1, all inputs were min-max scaled to ensure they fell within [0,1]. If inputs have not been scaled properly then they can cause the logistic function to saturate, and this can cause slow convergence when using gradient based methods (LeCun et al., 1998). The categorical variables do not have any inherent ordering, so to prevent this false ordering, one hot encoding was used. The Employed_days variable contains an integer 365243 which we interpret as applicants who aren't employed (there are 261 instances, or 16.9%). This would be an issue if left as is since it would dominate the scale, so was changed to 0. The Type_Occupation variable contained many sparse categories which was why the variable was removed to ensure that the input vector didn't become too large compared to the amount of training instances. Missing data was filled in by using the median (for numeric variables) and most common category (for categorical variables). Medians, mode, minimum and maximums were calculated using just the training instances. Scaling was applied to the validation and test instances, using the calculated values. They were also clipped between 0 and 1. This ensured no information was taken from these sets. The list of categorical values to be one hot encoded was chosen from the complete data set to ensure that the network has 32 inputs. This means that no values or statistics from the applicants in the validation and test sets were used.

4.2 Splitting with repeated applicants kept together (Objective 1)

There are many duplicates for the same applicants. Of the 1548 identifiers, 1526 have a recorded Birthday_count, and these contain only 1324 unique combinations of Birthday_count and Employed_days. There are 163 combinations that are duplicated across multiple identifiers, comprising 365 rows. In 93% of these groups all rows have the same label. We created a person key using 9 stable attributes (Car_Owner, Propert_Owner, Type_Income, EDUCATION, Marital_status, Housing_type, Employed_days, Family_Members and CHILDREN). This resulted in 1175 groups. This is a heuristic and does not guarantee two rows are the same person. All rows with the same key were always placed in the same split. We split the data into 70/15/15 training, validation and test sets. Whole groups were randomly assigned to each set and positives were proportionally represented. This resulted in 1083, 233 and 232 rows respectively. Both validation and test sets contained 26 or 27 positives. The random seed will affect the split. Different seeds result in different splits.

4.3 Network and training algorithm (Objective 2)

The network used (Rumelhart et al., 1986) has a feed forward structure with 32 input nodes, a single hidden layer containing h logistic nodes, and a single logistic output node. Given that u is the weighted sum at a hidden node, y is the output of a node, v is the weighted sum at the output node, z is the output of the node:

```
y_j = logistic(a_0j + sum over i of a_ij * x_i)
z = logistic(b_0 + sum over j of b_j * y_j)
logistic(u) = 1 / (1 + exp(-u)), set to 0 below -88 and to 1 above 88
p = (z - t) * z * (1 - z)      (derivative of the squared error with respect to v)
q_j = (p * b_j) * y_j * (1 - y_j)   (derivative of the squared error with respect to
u_j)
```

The bias derivatives are p and q_j. The weight derivatives are p y_j and q_j x_i. The weights are updated once per epoch (batch learning). Each weight is updated using the course update rule. The learning rate is increased by the amount κ (kappa) if the sign of the current derivative is the same as the average over the last few iterations f, otherwise it is reduced by a factor of φ (phi). The average is updated with a smoothing factor of θ (theta). The update is also combined with the last update using momentum μ (mu):

```
if d*f > 0 then e = e + κ else e = e * ϕ
f = (1 - θ)*d + θ*f
c = (1 - μ)*(-e*d) + μ*c
w = w + c
```

Where κ = 0.1; φ = 0.7; θ = 0.7; and μ = 0.8. The learning rate increases using an additive increase and multiplicative decrease such that it can increase when there is a consistent gradient in the direction of the derivative, and then rapidly decrease if the sign changes. This helps prevent oscillation. The average over the derivative (θ = 0.7) means that it is averaged over three epochs. Momentum smooths the updates. These parameters were used for all runs and were not optimised. The initial weights take the form given in the course program. The weights in the hidden layers were initialised to uniformly distribute between the values -0.1 and 0.1. The output weights alternate between +1 and -1. The weights are randomly sampled from a fixed seed.

The course program was adapted to allow the network to run with held out data. (i) The parameters are read from a file. (ii) The data file contains the training, validation and test examples. The network is run and the output for the validation and test examples are written to a file. (iii) Each example has a weight w_n applied to the p value. The derivatives are divided by the number of training examples meaning the κ parameter isn't dependent on the quantity of data. (iv) An optional parameter allows the weights to be reset to those with the smallest validation error for that epoch. The class weights were set to N/(2 n_c) so that both classes equally contribute to the error. This is a cost sensitive approach to address the 11.3% positive rate (He & Garcia, 2009). There were several errors in the course pseudocode. The loops for calculating the output used incorrect variables. One loop didn't have an initial value. And the hidden layer derivative had an x[j][n] instead of x[i][n]. These were corrected. The jump connection (input to output) was included in the program but wasn't used during the runs.

4.4 Experimental design (Objective 2)

There are three different configurations to compare (see Table 2). In runs A and B we have different numbers of hidden nodes. In run A there are 16 hidden nodes meaning there are 545 weights to train in total (32 inputs) and in run B there are 4 hidden nodes meaning there are only 137 weights to train (1083 rows in training set). The smaller network was used during the C1-C5 runs due to there being less weights to train compared to the larger network (137 weights compared to 545 weights for 1083 rows) therefore reducing the chance of overfitting. This was also because no real difference was seen in runs A & B. Runs A and B both use early-stopping on the validation error with 150 epochs patience. This is a common method to stop overfitting. However, in the validation set there were only 26 positives meaning the validation error was unstable (Section 5.2). Therefore, runs C1-C5 used a fixed number of epochs (300), with the final weights kept. Only the seed was varied, this affects the partitioning and initial weights. 300 epochs was chosen because looking at runs A and B we can see they stopped at 160 and 167 epochs respectively. 300 is approximately double this amount. This wasn't optimised.

Table 2. Experimental runs

Run	Hidden nodes	Training rule	Seed
A	16	early stopping, patience 150, best weights	0
B	4	early stopping, patience 150, best weights	0
C1	4	fixed 300 epochs, final weights	0
C2	4	fixed 300 epochs, final weights	1
C3	4	fixed 300 epochs, final weights	2
C4	4	fixed 300 epochs, final weights	3
C5	4	fixed 300 epochs, final weights	4

4.5 Evaluation (Objective 3)

The primary measure was the area under the ROC curve (AUC). This is the probability that a randomly selected positive example will have a higher output from the network than a randomly selected negative example. It does not depend on a threshold and is insensitive to the balance between classes (Fawcett, 2006). However, it can be noisy when there are few positives. Accuracy, precision, recall and F1 were also calculated from the test set confusion matrix. These require a decision threshold which was selected by choosing the threshold on the validation set that gave maximum F1. This threshold was then used without modification on the test set. As a baseline, predicting every application as rejected results in an F1 score of ~0.20 on the test set which contains 11.2% positives. Always predicting every application as approved results in an accuracy of ~0.89 but with a recall of 0. The results are quoted as the mean with the sample standard deviation.

5. Results and Discussion

5.1 Results

For Objective 1, the preprocessing produced 32 network inputs, and for every seed the split gave 1,083 training, 233 validation and 232 test rows, with 26 or 27 positives in each of the validation and test sets (Section 4.2)

Table 3. Seed 0: early stopping against fixed-length training (test set: 232 rows, 26 positives)

Run	Epochs trained (best)	Validation AUC	Test AUC	Test F1	Test recall	Test precision
A (16 hidden, early stopping)	160 (10)	0.460	0.569	0.207	0.885	0.117
B (4 hidden, early stopping)	167 (17)	0.569	0.493	0.154	0.308	0.103

C1 (4 hidden, 300 epochs)	300	0.462	0.528	0.208	1.000	0.116

On seed 0, the test AUC was 0.569 for run A, 0.493 for run B and 0.528 for run C1.

Table 4. Selected configuration over five splits (4 hidden nodes, 300 epochs, final weights)

Seed	Validation AUC	Test AUC	Threshold	Test accuracy	Test precision	Test recall	Test F1
0	0.462	0.528	0.130	0.147	0.116	1.000	0.208
1	0.683	0.660	0.580	0.728	0.190	0.407	0.259
2	0.562	0.548	0.479	0.487	0.129	0.593	0.212
3	0.624	0.657	0.634	0.841	0.238	0.192	0.213
4	0.488	0.618	0.356	0.362	0.128	0.808	0.221
Mean (SD)	0.564 (0.092)	**0.602 (0.061)**		0.513	0.160	0.600	0.223 (0.021)

The test AUC was above 0.5 in all five runs. With a standard error of 0.027, the 95% interval for the mean AUC is roughly 0.53 to 0.68. The five splits overlap in their data, so this interval should be read as approximate.

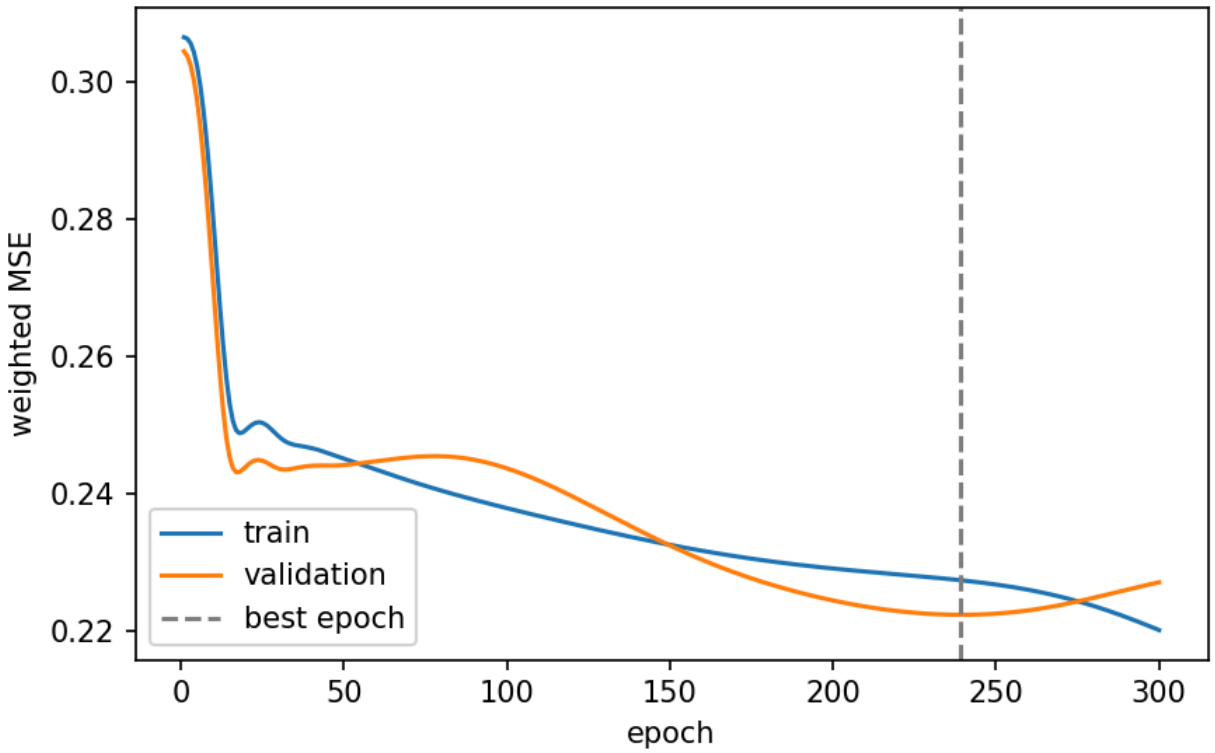


Figure 1. Weighted mean squared error per epoch for run C2 (4 hidden nodes, seed 1). The dashed line marks the epoch with the lowest validation error.

5.2 Discussion

What the network achieved: Overall, we built a network to predict outcomes for applications. Our network outperforms random guessing with an average AUC score of 0.602 on the test set. So,if we pick a rejected applicant at random and compare them to a randomly picked approved applicant, our model will give a higher output value (a higher score for rejection) to the rejected applicant about 60% of the time. This is a small signal but is still present. However, it is not high enough to make decisions automatically. In run C2 (Figure 1) the training error reduced consistently from approximately 0.31 to 0.22. The error on the validation set reduced quickly in the first 20 epochs. It then reached a minimum at around epoch 239 before increasing slightly, which suggests that the network had learned what it could from the data and was beginning to overfit.

Variability: For the same setting, we obtained an AUC value between 0.528 and 0.660 (SD = 0.061) based on different seeds. As the positive applications in the test sets are few (only 26), there is some variance in our results. As such, the results from individual runs cannot be interpreted and the mean across the splits should be considered.

Threshold-dependent metrics: The accuracy varied between 14.7% and 84.1% and recall varied between 0.192 and 1.000. This is because the threshold varied between 0.130 and 0.634

on the validation set. Since there are only 26 positives in the validation set, the threshold will vary quite a bit. The average F1 score was 0.223, which is not much of an improvement on the 0.20 obtained when every application is predicted as rejected . This means F1 and recall do not provide much additional information over the AUC. Therefore, only the AUC should be relied on. The lowest validation error was achieved at epoch 17 in 3 of the runs, and at epochs 132 and 239 in the other 2 runs. This suggests that it would be better to train the model for a fixed amount of time rather than ending when the validation error has reached its minimum since the validation set is so small.

Network size and training rule (Objective 2). Runs A, B and C1 used the same seed and therefore the same split. The resulting test (AUC) values were 0.569, 0.493 and 0.528 and the validation AUC values were 0.460, 0.569 and 0.462 respectively. These differences are within the range of the sampling noise described above and therefore cannot be used to determine an ordering of network sizes or training rules. This small improvement over random guessing and lack of any marked improvements with increasing network size are consistent with the observation that credit data is often only slightly non-linear (Baesens et al., 2003). However, this cannot be confirmed by this study. The jump connections were implemented but were not tested.

Repeated applicants. There were duplicates within the data (Section 4.2). Using random splits would have resulted in the same individual appearing in both the training and test sets, which may artificially inflate results. Splits were therefore designed using groups of rows with the same Person Key. This study did not quantify how the grouping affected the results so we can say only that the grouped design reduces this risk.

Limitations. The person key is a heuristic. It doesn't prove that rows belong to the same individual. There are only 26/27 positives within each test set meaning that all results have limited precision. We chose to run 300 epochs based on our early stopping results. We have used one seed for both 4 and 16 hidden nodes. The data does not contain typical credit scoring data that would be most predictive, such as repayment history.

6. Conclusion and Recommendations

Conclusion: The feed forward network designed using back propagation algorithm from the course was compiled using QB64. It was able to identify a weak but real signal from the credit application data. Using four hidden nodes and 300 epochs of training, the model had an average test AUC score of 0.602 (SD = 0.061) across five splits with repeated applications grouped together. This was higher than chance in all instances. However, due to instability in the threshold, the accuracy, precision, recall and F1 scores are not suitable to imply practicality. Due to the low number of positive cases, the results from the different network sizes and training rules were too similar to provide accurate rankings. This study highlights that the way in which models are evaluated is an important consideration alongside how the networks are created. Grouping repeated applications, weighting classes and reporting AUC were all relevant considerations.

Recommendations:

1. Use official identifiers instead of heuristics for individuals.
2. Create a larger set of positive examples with more informative features (such as repayment history).
3. Perform multiple grouped cross validations in order to get more accurate performance estimates.
4. Compare the network with different network architectures (such as a jump-connected network without hidden layer, which is equivalent to logistic regression) and other families of models to determine if the limitation comes from the network itself or the data.
5. Select the decision threshold based on the costs associated with a false positive (an approved applicant predicted as rejected) or a false negative (a rejected applicant predicted as approved). Validate on a larger validation set.

7. References

Baesens, B., Van Gestel, T., Viaene, S., Stepanova, M., Suykens, J., & Vanthienen, J. (2003). Benchmarking state-of-the-art classification algorithms for credit scoring. *Journal of the Operational Research Society*, 54(6), 627–635. https://doi.org/10.1057/palgrave.jors.2601545

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. https://doi.org/10.1109/TKDE.2008.239

LeCun, Y., Bottou, L., Orr, G. B., & Müller, K.-R. (1998). Efficient BackProp. In G. B. Orr & K.-R. Müller (Eds.), *Neural Networks: Tricks of the Trade* (Lecture Notes in Computer Science, Vol. 1524, pp. 9–50). Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-49430-8_2

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. https://doi.org/10.1038/323533a0

8. Appendix

A. Source code. [https://github.com/aiancutines/Cutines_AC_CMSC191.git]

- **backprop.bas**: The course backpropagation program (QB64), extended as described in Section 4.3.
- **test.py**: Builds the data file PROBLEM (preprocessing and grouped split) and scores the program's output.
- **network.txt**: Run settings, one value per line.
- **PROBLEM**: Generated data file with training, validation, and test rows.

B. Settings files. Each run used a network.txt with these 11 lines in order: hidden nodes, kappa, phi, theta, mu, jump connections (0 or 1), maximum epochs, patience, seed, split type (0 = repeated applicants kept together), restore best weights (1 = yes, 0 = keep final weights).

Run	network.txt (top to bottom)
A	16, 0.1, 0.7, 0.7, 0.8, 0, 2000, 150, 0, 0, 1
B	4, 0.1, 0.7, 0.7, 0.8, 0, 2000, 150, 0, 0, 1
C1	4, 0.1, 0.7, 0.7, 0.8, 0, 300, 300, 0, 0, 0
C2	4, 0.1, 0.7, 0.7, 0.8, 0, 300, 300, 1, 0, 0
C3	4, 0.1, 0.7, 0.7, 0.8, 0, 300, 300, 2, 0, 0
C4	4, 0.1, 0.7, 0.7, 0.8, 0, 300, 300, 3, 0, 0
C5	4, 0.1, 0.7, 0.7, 0.8, 0, 300, 300, 4, 0, 0

C. How to reproduce. Compile backprop.bas in QB64 (Run, Make EXE Only). For each run, put that run's values in network.txt, run python test.py to build PROBLEM, run backprop.exe, then run python test.py score, which prints the metrics and appends them to results_log.csv.